PassDown

Software Requirements Specification

Final submission draft — Version 1.0

Course	Software Engineering
Institution	Independent University, Bangladesh (IUB)
Project	PassDown—Academic Resource-Sharing Platform
Submission date	October 2026

Team information

Role	Full name	Student ID
Member 1	Tanjil Hasan Emon	2331024
Member 2	Tanjil Sarkar	2312026
Member 3	Md Jawad Abdullah	2230325
Member 4	S M Mashraful Islam	2230836

Contents

1. Introduction **1**
1.1 Purpose . . . 1
1.2 Intended audience . . . 1
1.3 Intended use . . . 1
1.4 Product scope . . . 2
In scope for the MVP . . . 2
Goals . . . 2
Explicitly out of scope for the MVP . . . 2
1.5 Definitions and abbreviations . . . 2
1.6 Risk definitions . . . 3

2. Overall description **4**
2.1 Product perspective . . . 4
2.2 User classes and characteristics . . . 5
2.3 User needs . . . 6
2.4 Operating environment . . . 6
2.5 Constraints . . . 6
2.6 Assumptions and dependencies . . . 6
2.7 Approved design decisions . . . 7

3. Requirements **7**
3.1 Functional requirements — user stories with confirmation . . . 7
FR-01 — Register and view a user profile . . . 8
FR-02 — Authenticate and authorize protected sessions . . . 8
FR-03 — Browse and search courses . . . 9
FR-04 — Manage courses . . . 9
FR-05 — Browse and filter public materials . . . 10
FR-06 — Manage Core Materials . . . 11
FR-07 — Submit a Contribution . . . 11
FR-08 — Moderate Contributions . . . 12
FR-09 — Manage one's own pending Contribution . . . 12
FR-10 — Submit a Request . . . 13
FR-11 — Browse Requests . . . 14
FR-12 — Fulfill a Request . . . 14
3.2 Non-functional requirements . . . 15
3.3 External interface requirements . . . 17
3.3.1 User interface . . . 17
3.3.2 REST API . . . 17

3.3.3 Data interface 19
3.3.4 External services and hosting 19
3.4 Business rules and status transitions 20
Roles and scopes 20
Visibility and status rules 20
Input validation 21
3.5 Traceability and verification matrix 21

4. Verification and release acceptance **23**
4.1 Test strategy 23
4.2 Minimum role/permission test matrix 23
4.3 Definition of Done 24
4.4 Demonstration sequence 24

1. Introduction

1.1 Purpose

PassDown is a web-based academic resource-sharing platform for university students. It provides a reliable, course-wise catalogue of external links to learning materials and gives students a structured way to contribute materials or request resources they cannot find.

This SRS defines **what PassDown must do**, the quality and security constraints it must satisfy, the actors and permissions involved, and the confirmation criteria that will be used for implementation and testing. It is the shared reference for the product team, developers, testers, demonstrators, and course evaluator.

The system stores links and metadata; it does **not** upload or host files directly.

1.2 Intended audience

Audience	How this document is used
Course instructor/evaluator	Review scope, quality, security, and evidence of requirements satisfaction.
Team Lead	Control scope, approve changes, assign Trello cards, and verify release readiness.
Developers	Implement API, database, authentication, UI, and deployment behavior.
Test/QA owners	Convert confirmation criteria into automated and manual tests.
Student users	Understand the supported workflows and visibility rules.
Future maintainers	Understand the architecture, decisions, limitations, and deferred work.

1.3 Intended use

The team shall use this SRS to:

1. divide the MVP into exactly 12 primary functions;
2. create one or more Trello cards for each function;
3. implement only behavior that has a requirement or an approved change record;
4. derive API tests, UI tests, security tests, and demonstration scenarios;

5. review scope during weekly meetings; and
6. prove completion through traceability and evidence rather than verbal claims.

Discord is used for communication and decisions. Trello is the official source of task status. Git pull requests and test reports are the technical evidence.

1.4 Product scope

In scope for the MVP

- Public browsing of courses and public resource information.
- Student registration and login using hashed passwords and JWT-based authentication.
- Role-based access control for `admin` and `student` users, with unauthenticated `guest` access to public features.
- Admin management of courses and Core Materials.
- Student-submitted Contributions that remain hidden until moderation.
- Admin approval or rejection of Contributions.
- Student Requests for missing course materials and a fulfillment workflow.
- Course search by code or title and material filtering by type.
- Responsive React/TypeScript frontend and FastAPI REST backend.
- PostgreSQL persistence through SQLAlchemy, deployable on the course-approved free-tier architecture.

Goals

1. Give students one dependable place to preview courses and find useful materials.
2. Preserve a trusted baseline through admin-curated Core Materials.
3. Allow community participation without publishing unreviewed links.
4. Demonstrate a complete full-stack application with authentication, RBAC, security scopes, REST APIs, layered architecture, and meaningful tests.

Explicitly out of scope for the MVP

- File upload, file hosting, or storage of course documents.
- Real-time chat, direct messaging, or notifications.
- AI/ML recommendations or semantic search.
- Payments, subscriptions, or premium tiers.
- Native mobile applications.
- Comments, votes, thanks counters, contributor leaderboard, and automated URL reachability checks.

1.5 Definitions and abbreviations

Term	Definition
Admin	A trusted platform curator provisioned directly in the database or through a controlled seed process.
Student	An authenticated registered user with the default application role.
Guest	An unauthenticated visitor with read-only access to public content.
Core Material	An admin-curated external resource link that is publicly visible.
Contribution	A user-submitted external resource link that begins as `pending` and requires moderation.
Request	A course-specific post asking the community for a missing resource.
RBAC	Role-Based Access Control.
JWT	JSON Web Token used to represent an authenticated session.
Scope	A fine-grained permission such as `courses:write` or `contributions:moderate`.
MVP	Minimum Viable Product defined by the approved scope in this document.
Confirmation part	Given/When/Then acceptance criteria proving that a user story is satisfied.

1.6 Risk definitions

Risk	Impact	Mitigation and confirmation
Unreviewed or malicious links become public	High	Contributions are `pending` by default; only Admin approval makes them public. Validate URL syntax and test the visibility rule.
Unauthorized user performs an admin or ownership action	High	Use JWT validation, FastAPI security scopes, service-layer ownership checks, and explicit 401/403 tests.

Risk	Impact	Mitigation and confirmation
Passwords or deployment secrets leak	High	Hash passwords, use environment variables, ignore `.env`, prohibit secrets in Discord/Trello/Git, and review the repository before release.
Course deletion removes dependent records unexpectedly	High	MVP policy: block deletion when dependent records exist and return `409 Conflict`; do not cascade-delete student data.
Requirements are interpreted differently by team members	Medium	Use this SRS as the baseline, record decisions in Trello and Discord, and require acceptance criteria on every feature card.
Free-tier serverless cold start delays the first request	Medium	Document it as an accepted limitation and measure warm-request performance separately from cold-start behavior.
External links become unavailable	Medium	Store the source URL and metadata; automated reachability checking is explicitly deferred. Report this limitation to users rather than pretending links are guaranteed.
Scope expands beyond the semester	High	Keep deferred ideas in the Trello `Deferred / Future` list. Require Team Lead approval for scope changes.

2. Overall description

2.1 Product perspective

PassDown is a standalone full-stack web application with a React/TypeScript single-page frontend, a FastAPI REST backend, and a PostgreSQL database. The recommended course-compatible deployment is one Vercel project serving the frontend and `/api/*` backend routes, with Supabase providing managed PostgreSQL.

```
Browser
  | HTTPS
  v
React + TypeScript SPA -- /api/v1 --> FastAPI routers
                                          |
                                          v
                                   Services / RBAC
                                          |
                                          v
                                   Repositories / ORM
                                          |
                                          v
                                   PostgreSQL (Supabase)
```

The backend shall follow the layered design described in the TDD:

```
Routers --> Services --> Repositories --> SQLAlchemy/PostgreSQL
   |          |
   +----------+-- Pydantic schemas, dependencies, security scopes
```

Routers handle HTTP concerns, services enforce business rules, repositories perform database access, and schemas validate/shape data. The frontend shall not be treated as a security boundary; every protected action must also be enforced by the backend.

2.2 User classes and characteristics

User class	Characteristics	Main needs
Guest	Unauthenticated visitor; may be a student deciding whether to take a course.	Search courses, inspect Core Materials, view approved Contributions, and view Requests without creating data.
Student	Registered and authenticated user.	Log in securely, submit Contributions, manage own pending Contributions, submit Requests, and fulfill own Requests.
Admin	Trusted curator provisioned outside public registration.	Manage courses and Core Materials, moderate Contributions, fulfill/delete Requests where authorized, and maintain content quality.

2.3 User needs

1. A student needs to preview a course before enrolling or preparing for it.
2. A student needs trustworthy, organized resource links instead of relying only on personal contacts.
3. A student needs a safe way to ask for missing materials.
4. An admin needs a manageable moderation queue and control over the course catalogue.
5. The evaluator needs observable evidence of client-server integration, authentication, RBAC, security scopes, database persistence, and testing.

2.4 Operating environment

- Modern desktop and mobile browsers with JavaScript enabled.
- Frontend: Vite + React + TypeScript.
- Backend: Python + FastAPI under `/api/v1`.
- Database: PostgreSQL through SQLAlchemy; Supabase is the target managed service.
- Development: backend on port `8000`, frontend on port `5173`, or the course template's equivalent combined development command.
- Production: HTTPS deployment using the approved Vercel/Supabase arrangement.

2.5 Constraints

- Zero-budget/free-tier infrastructure.
- No direct file storage; only external HTTPS URLs and metadata are stored.
- Admin accounts are not created through public registration.
- The MVP targets a single university context but must not hard-code a design that prevents later expansion.
- The team must preserve the course template's React + TypeScript and FastAPI + Python stack.
- The team must demonstrate REST APIs, authentication, RBAC, and security scopes.

2.6 Assumptions and dependencies

- Students can host files in services such as Google Drive, OneDrive, GitHub, or another permitted external provider.
- The database is reachable through a configured environment variable and is available during demonstrations.
- The instructor accepts the selected deployment architecture and the defined 12-function grouping.
- Team members have access to the shared repository, Discord server, Trello board, and deployment project.

- External link reachability is not guaranteed by the MVP because automated link validation is deferred.

2.7 Approved design decisions

The following decisions establish the approved MVP baseline and remove inconsistencies across the supplied PRD, SRS, and TDD.

Decision	Recommended MVP baseline
Deployment target	Use one Vercel project for the frontend and FastAPI routes, with Supabase PostgreSQL, as specified in the TDD.
Admin submissions	Permit an authenticated Admin to submit Contributions and Requests because the approved permission matrix and TDD scopes allow it.
Public Request browsing	Allow Guests to view Requests; Request browsing is a public read feature.
Admin user management	Defer it from this MVP because no endpoint, schema, or UI is specified.
Course deletion	Allow deletion only when no dependent Materials, Contributions, or Requests exist; otherwise return `409 Conflict`. This avoids accidental cascade deletion.
Semester representation	Use one validated `semester_offered` string for the MVP, for example `Spring 2027`. Change the schema before implementation if multiple semester records are required.
JWT storage	Keep the access token in frontend memory and attach it as a Bearer token; do not store it in `localStorage`. Clear session state on `401`.

---

3. Requirements

3.1 Functional requirements — user stories with confirmation

The following are the **exact 12 primary functions** for the project. Every function has a user story and a confirmation part. The confirmation criteria are also the minimum acceptance criteria for the related Trello card and test case.

FR-01 — Register and view a user profile

User story: As a prospective student, I want to register with my name, email, and password and view my own profile, so that I can use protected PassDown features.

Actors: Guest registering; authenticated Student or Admin viewing `/me`.
Authorization: Registration is public; profile retrieval requires a valid authenticated session.

Confirmation / acceptance criteria:

- **Given** a visitor submits a unique valid email, name, and password, **when** registration is submitted, **then** the system creates one account with role `student` and stores only a secure password hash.
- **Given** the submitted email already exists or required data is invalid, **when** registration is submitted, **then** the system returns a clear validation/conflict error and creates no duplicate account.
- **Given** a public registration request attempts to set `role=admin`, **when** it is processed, **then** the system rejects or ignores that value and does not create a public Admin account.
- **Given** an authenticated user, **when** the user requests `/api/v1/auth/me`, **then** the response contains the user id, name, email, role, and creation information allowed by the response schema, but never the password hash.

Primary endpoints: `POST /api/v1/auth/register`, `GET /api/v1/auth/me`
Primary evidence: registration integration test, duplicate-email test, role-escalation test, `/me` response test.

FR-02 — Authenticate and authorize protected sessions

User story: As a registered user, I want to log in and receive an expiring authenticated session, so that the system can authorize my actions according to my role and scopes.

Actors: Student and Admin.
Authorization: Login is public; protected endpoints require a valid token and required security scopes.

Confirmation / acceptance criteria:

- **Given** valid registered credentials, **when** the login form is submitted, **then** the system returns a signed JWT containing the subject identity, role/scopes, and expiry.
- **Given** invalid credentials, **when** login is submitted, **then** the system returns `401 Unauthorized` and issues no usable token.

- **Given** a missing, invalid, or expired token, **when** a protected endpoint is called, **then** the system returns `401 Unauthorized`.
- **Given** a valid token without the endpoint's required scope, **when** the endpoint is called, **then** the system returns `403 Forbidden` and changes no data.
- **Given** the frontend receives `401`, **when** the response interceptor handles it, **then** it clears the in-memory session and provides a path to log in again.

Primary endpoint: `POST /api/v1/auth/login`
Primary evidence: login test, expiry/invalid-token test, Student-versus-Admin authorization matrix, frontend session test.

FR-03 — Browse and search courses

User story: As a Guest, Student, or Admin, I want to browse and search courses by code or title, so that I can find the course whose materials I need.

Actors: Guest, Student, Admin.
Authorization: Public read access.

Confirmation / acceptance criteria:

- **Given** a public visitor requests the course list, **when** the request succeeds, **then** the response includes course code, title, department, offered semester, and stable identifiers.
- **Given** a search term matches a course code or title, **when** `?search=` is supplied, **then** matching courses are returned without requiring login.
- **Given** a valid course id, **when** its detail is requested, **then** the course information is returned with links or navigation to its public resource sections.
- **Given** an unknown course id, **when** its detail is requested, **then** the system returns a defined `404 Not Found` response.

Primary endpoints: `GET /api/v1/courses`, `GET /api/v1/courses/{id}`
Primary evidence: guest browser test, search test, not-found test, responsive course-list screenshot.

FR-04 — Manage courses

User story: As an Admin, I want to create, edit, and delete course records, so that the public catalogue remains accurate and curated.

Actors: Admin; Guest/Student as denied callers.
Authorization: Admin scope `courses:write`.

Confirmation / acceptance criteria:

- **Given** an authenticated Admin submits valid unique course data, **when** creation is requested, **then** the course is persisted and appears in public browsing.
- **Given** an existing course and valid replacement data, **when** an authorized Admin updates it, **then** the new values are persisted and visible in subsequent reads.
- **Given** a course has no dependent records, **when** an authorized Admin deletes it, **then** it is removed from public browsing.
- **Given** a course has Core Materials, Contributions, or Requests, **when** deletion is requested, **then** the system returns `409 Conflict` and preserves the dependent data.
- **Given** a Guest or Student attempts a mutation, **when** the request is sent, **then** the system returns `401` or `403` and makes no change.

Primary endpoints: `POST /api/v1/courses`, `PUT /api/v1/courses/{id}`, `DELETE /api/v1/courses/{id}`
Primary evidence: Admin CRUD test, duplicate-code test, dependency-protection test, Student/Guest denial test.

FR-05 — Browse and filter public materials

User story: As a visitor or authenticated user, I want to browse a course's Core Materials and approved Contributions and filter them by type, so that I can find relevant links without seeing unreviewed content.

Actors: Guest, Student, Admin.
Authorization: Public read access; only approved Contributions are public.

Confirmation / acceptance criteria:

- **Given** a valid course, **when** public materials are requested, **then** the response contains Core Materials and approved Contributions with title, type, description, external URL, and source/category information.
- **Given** a Contribution is `pending` or `rejected`, **when** public materials are requested, **then** that Contribution is excluded.
- **Given** a supported type filter such as `slides`, `notes`, `past_paper`, `book`, or `other`, **when** it is applied, **then** only matching material records are returned.
- **Given** no records match the selected filter, **when** the page renders, **then** the UI shows a clear empty state rather than an error or misleading result.

Primary endpoints: `GET /api/v1/courses/{id}/core-materials`, `GET /api/v1/courses/{id}/contributions` **Primary evidence:** public visibility test, type-filter test, pending/rejected exclusion test, UI empty-state test.

FR-06 — Manage Core Materials

User story: As an Admin, I want to create, edit, and delete course-linked Core Material links, so that every course has a trusted baseline of resources.

Actors: Admin; Guest/Student as denied callers.
Authorization: Admin scope `materials:write`.

Confirmation / acceptance criteria:

- **Given** an existing course and valid title, type, URL, and description, **when** an authorized Admin submits the material, **then** one Core Material is persisted and associated with that course.
- **Given** an existing Core Material, **when** an authorized Admin edits it, **then** the updated metadata appears in the public course view.
- **Given** an existing Core Material, **when** an authorized Admin deletes it, **then** it no longer appears in public results.
- **Given** the course does not exist or the URL/type is invalid, **when** creation is attempted, **then** the system returns a validation/not-found error and creates no orphaned record.
- **Given** a Guest or Student attempts a Core Material mutation, **when** the request is sent, **then** the system returns `401` or `403` and makes no change.

Primary endpoints: `POST /api/v1/courses/{id}/core-materials`, `PUT /api/v1/core-materials/{id}`, `DELETE /api/v1/core-materials/{id}` **Primary evidence:** Admin CRUD test, course foreign-key test, URL/type validation test, denial test.

FR-07 — Submit a Contribution

User story: As an authenticated Student, or an Admin acting as an authorized contributor, I want to submit an external material link for a course, so that the community can review and publish useful resources.

Actors: Student and Admin.
Authorization: Scope `contributions:write`; Guests are denied.

Confirmation / acceptance criteria:

- **Given** an authorized authenticated user submits a valid course, title, type, URL, and description, **when** the Contribution is created, **then** it is persisted with status `pending` and the submitting user is recorded.
- **Given** a Guest or authenticated user without `contributions:write`, **when** submission is attempted, **then** the system returns `401` or `403` and creates no record.
- **Given** the course is invalid or required material data is invalid, **when** submission is attempted, **then** the system returns a validation/not-found response and creates no

record.

- **Given** a newly submitted Contribution is still `pending`, **when** public materials are requested, **then** it is not visible to Guests or ordinary Students.

Primary endpoint: `POST /api/v1/courses/{id}/contributions`
Primary evidence: authenticated submission test, default-status test, invalid-input test, Guest denial test.

FR-08 — Moderate Contributions

User story: As an Admin, I want to review, approve, or reject pending Contributions, so that only curated links become publicly visible.

Actors: Admin; Student/Guest as denied callers.
Authorization: Scope `contributions:moderate`.

Confirmation / acceptance criteria:

- **Given** an authenticated Admin requests the moderation queue, **when** the request succeeds, **then** all and only the appropriate pending Contributions are returned with enough metadata for review.
- **Given** a pending Contribution, **when** an authorized Admin approves it, **then** its status changes to `approved` and it appears in public material results.
- **Given** a pending Contribution, **when** an authorized Admin rejects it, **then** its status changes to `rejected` and it remains hidden from public users.
- **Given** a Student or Guest attempts to view the moderation queue or change a status, **when** the request is sent, **then** the system returns `403` (or `401` without a token) and leaves the Contribution unchanged.
- **Given** a Contribution is already approved or rejected, **when** a second incompatible moderation action is attempted, **then** the system follows the documented status-transition rule and returns a clear response.

Primary endpoints: `GET /api/v1/contributions/pending`, `PATCH /api/v1/contributions/{id}/approve`, `PATCH /api/v1/contributions/{id}/reject` **Primary evidence:** moderation queue test, approve visibility test, reject visibility test, under-privileged denial test.

FR-09 — Manage one's own pending Contribution

User story: As a Student who submitted a Contribution, I want to edit or withdraw it while it is pending, so that I can correct mistakes before moderation.

Actors: Contribution owner; Admin may delete according to moderation policy.

Authorization: Owner checks are enforced in the service layer; Student changes are limited to `pending` records.

Confirmation / acceptance criteria:

- **Given** a Student owns a pending Contribution, **when** valid edits are submitted, **then** the metadata is updated and the status remains `pending`.
- **Given** a Student owns a pending Contribution, **when** deletion is requested, **then** the Contribution is removed and is not publicly visible.
- **Given** a Student does not own the Contribution, **when** edit or deletion is attempted, **then** the system returns `403` or `404` according to the chosen privacy policy and makes no change.
- **Given** a Contribution is already approved or rejected, **when** its Student owner tries to edit it, **then** the system rejects the operation because student editing is pending-only.
- **Given** an Admin deletes a Contribution under the documented policy, **when** the deletion succeeds, **then** the record is absent from future public and moderation results.

Primary endpoints: `PUT /api/v1/contributions/{id}`, `DELETE /api/v1/contributions/{id}`
Primary evidence: owner-success test, non-owner denial test, status-boundary test, Admin deletion test.

FR-10 — Submit a Request

User story: As an authenticated Student, or an Admin acting as an authorized requester, I want to describe a missing material for a course, so that the community knows what resource is needed.

Actors: Student and Admin.
Authorization: Scope `requests:write`; Guests are denied.

Confirmation / acceptance criteria:

- **Given** an authorized authenticated user submits a valid course and non-empty description, **when** the Request is created, **then** it is persisted with status `open` and the submitting user is recorded.
- **Given** a Guest or authenticated user without `requests:write`, **when** submission is attempted, **then** the system returns `401` or `403` and creates no record.
- **Given** the course is invalid or the description is missing/blank, **when** submission is attempted, **then** the system returns a validation/not-found response and creates no record.
- **Given** a new Request has been created, **when** it is listed for the course, **then** its open status and description are visible as a public read feature.

Primary endpoint: `POST /api/v1/courses/{id}/requests`
Primary evidence: request creation test, default-status test, validation test, Guest denial test.

FR-11 — Browse Requests

User story: As a Guest, Student, or Admin, I want to view Requests for a course and see whether each is open or fulfilled, so that I can identify unmet needs.

Actors: Guest, Student, Admin.
Authorization: Public read access under the decision in §2.7.

Confirmation / acceptance criteria:

- **Given** a valid course, **when** its Requests are requested, **then** the response includes request id, description, status, author display information allowed by policy, and creation information.
- **Given** a Request is `open` or `fulfilled`, **when** the public list is loaded, **then** its current status is represented consistently in the API and UI.
- **Given** a Guest requests a course's Requests, **when** the request succeeds, **then** no login is required and no write control is exposed.
- **Given** an unknown course id, **when** its Requests are requested, **then** the system returns a defined `404 Not Found` response.

Primary endpoint: `GET /api/v1/courses/{id}/requests`
Primary evidence: Guest list test, status-rendering test, not-found test, UI read-only test.

FR-12 — Fulfill a Request

User story: As the author of a Request or an Admin, I want to mark an open Request as fulfilled, so that the catalogue reflects that the need has been addressed.

Actors: Request owner and Admin; other Students/Guests are denied.
Authorization: Owner-or-Admin rule plus scope `requests:write`; ownership is checked in the service layer.

Confirmation / acceptance criteria:

- **Given** the original author owns an open Request, **when** fulfillment is submitted, **then** the status changes to `fulfilled` and the updated status is returned.
- **Given** an Admin with the required authority submits fulfillment for an open Request, **when** the request succeeds, **then** the status changes to `fulfilled`.

- **Given** another Student, a Guest, or an unauthenticated caller attempts fulfillment, **when** the request is sent, **then** the system returns `401` or `403` and leaves the Request unchanged.
- **Given** a Request is already fulfilled, **when** fulfillment is submitted again, **then** the system returns a clear idempotent or conflict response according to the documented API rule.
- **Given** the TDD `DELETE /requests/{id}` endpoint is retained, **when** the owner or Admin deletes a Request under the approved policy, **then** it is removed without affecting the course or materials.

Primary endpoints: `PATCH /api/v1/requests/{id}/fulfill`, `DELETE /api/v1/requests/{id}`
Primary evidence: owner-success test, Admin-success test, non-owner denial test, repeated-fulfillment test.

3.2 Non-functional requirements

ID	Category	Requirement and confirmation method
NFR-01	Security	Passwords shall be hashed with a suitable password-hashing algorithm and shall never be stored, returned, or logged in plaintext. Confirm with a database inspection and response/log review.
NFR-02	Security	JWTs shall be signed with a secret loaded from an environment variable, include expiry, and be rejected when invalid or expired. Confirm with automated authentication tests.
NFR-03	Authorization	Protected endpoints shall enforce authentication, role/scopes, and data-dependent ownership in the backend. Confirm with a Guest/Student/Admin matrix containing both success and 401/403 cases.
NFR-04	Configuration	Production CORS shall allow only the deployed frontend origin; development origins shall be explicit. Secrets, database URLs, and credentials shall not be committed. Confirm through configuration review and repository scanning.

ID	Category	Requirement and confirmation method
NFR-05	Performance	With a representative seeded dataset (for example, up to 1,000 courses and 10,000 material/request records), public browse/search endpoints should achieve p95 response time of **2 seconds or less** for 20 warm sequential requests in the agreed test environment. Cold starts shall be recorded separately as a known limitation.
NFR-06	Usability	Core flows shall provide clear navigation, form labels, validation messages, loading states, empty states, and actionable error messages. Confirm through the manual browser checklist.
NFR-07	Responsiveness	Public browsing and protected forms shall remain usable at desktop and mobile viewport widths without horizontal scrolling or inaccessible controls. Confirm with at least one desktop and one mobile-width browser check.
NFR-08	Reliability	API failures shall return consistent JSON error shapes and appropriate HTTP status codes; invalid writes shall not create partial/orphaned records. Confirm with negative integration tests and database checks.
NFR-09	Maintainability	Backend code shall preserve the routers → services → repositories → schemas separation. Shared role/scope names and API types shall be documented and reused rather than duplicated inconsistently. Confirm by code review.
NFR-10	Data integrity	Course relationships, unique course codes/emails, allowed enum/status values, timestamps, and ownership fields shall be validated by the schema and database. Confirm with constraint and service tests.
NFR-11	Portability	The application shall run with documented environment variables on the course template locally and on the approved free-tier deployment without code changes that expose secrets. Confirm with a clean setup/deployment checklist.
NFR-12	Quality and evidence	Every implemented function shall have at least one automated or repeatable manual confirmation, a linked Trello card, a reviewed pull request, and evidence attached before it is marked Done. Confirm through the traceability matrix.

3.3 External interface requirements

3.3.1 User interface

The frontend shall provide the following minimum routes or equivalent screens:

Area	Required screens and behavior
Public	Home, course list, course detail, Core Materials, approved Contributions, and Requests.
Authentication	Register, Login, and a clear unauthorized/expired-session state.
Student	Submit Contribution, own pending Contributions, Submit Request, and own Requests/fulfillment controls.
Admin	Manage Courses, Manage Core Materials, and Moderate Contributions. Admin-only controls shall not be shown as actionable to Guests or Students.
Shared UI	Navigation, responsive layout, form validation, loading states, empty states, error messages, and external-link affordances.

The UI may hide controls for convenience, but hiding a control is not authorization. The backend must repeat every permission check.

3.3.2 REST API

Base path: `/api/v1`. All request and response bodies shall use JSON except where the OAuth2 password-flow implementation requires the template's documented form encoding for login.

Method	Endpoint	Access	Purpose
`POST`	`/auth/register`	Public	Create a default Student account.
`POST`	`/auth/login`	Public	Verify credentials and return a JWT.
`GET`	`/auth/me`	Authenticated	Return the current user's safe profile.

Method	Endpoint	Access	Purpose
GET	/courses	Public	List courses; support ?search=.
GET	/courses/{id}	Public	Return course detail.
POST	/courses	Admin	Create a course.
PUT	/courses/{id}	Admin	Update a course.
DELETE	/courses/{id}	Admin	Delete a course only when the dependency policy permits it.
GET	/courses/{id}/core-materials	Public	List Core Materials, with the agreed type filter.
POST	/courses/{id}/core-materials	Admin	Add a Core Material.
PUT	/core-materials/{id}	Admin	Edit a Core Material.
DELETE	/core-materials/{id}	Admin	Delete a Core Material.
GET	/courses/{id}/contributions	Public	List approved Contributions only.
GET	/contributions/pending	Admin	List pending Contributions for moderation.
POST	/courses/{id}/contributions	Student/Admin	Submit a pending Contribution.
PATCH	/contributions/{id}/approve	Admin	Approve a pending Contribution.
PATCH	/contributions/{id}/reject	Admin	Reject a pending Contribution.
PUT	/contributions/{id}	Owner, pending only	Edit an owned pending Contribution.
DELETE	/contributions/{id}	Owner/Admin per policy	Delete a Contribution.
GET	/courses/{id}/requests	Public	List Requests for a course.
POST	/courses/{id}/requests	Student/Admin	Create an open Request.

Method	Endpoint	Access	Purpose
`PATCH`	`/requests/{id}/fulfill`	Owner/Admin	Mark an open Request fulfilled.
`DELETE`	`/requests/{id}`	Owner/Admin under the approved deletion policy	Delete a Request under the approved policy.

The exact request schemas, response schemas, error shape, and status codes shall be documented in the repository's API documentation or OpenAPI output and linked from the Trello project-control card.

3.3.3 Data interface

The minimum relational entities are:

Entity	Important fields and rules
`User`	`id`, `name`, unique `email`, `password_hash`, role, `created_at`.
`Course`	`id`, unique `code`, `title`, `department`, `semester_offered`, `created_at`.
`CoreMaterial`	`id`, `course_id`, `title`, allowed `type`, validated external `url`, optional `description`, `created_at`.
`Contribution`	`id`, `course_id`, `submitted_by`, `title`, allowed `type`, `url`, optional `description`, status, `created_at`.
`Request`	`id`, `course_id`, `submitted_by`, non-empty `description`, status, `created_at`.

Foreign keys and unique constraints shall be enforced at the database or ORM boundary where practical. Service-layer rules shall enforce ownership and status transitions.

3.3.4 External services and hosting

- Supabase PostgreSQL is the target database service.

- Vercel is the recommended combined frontend/backend deployment target from the TDD.
- External storage providers are link destinations only; PassDown does not use their APIs or store their files.
- Environment variables such as `DATABASE_URL`, `JWT_SECRET`, and `CORS_ORIGINS` shall be configured outside source control.

3.4 Business rules and status transitions

Roles and scopes

Roles and fine-grained scopes are separate concepts:

Role	Minimum scopes
`admin`	`courses:write`, `materials:write`, `contributions:moderate`, `contributions:write`, `requests:write`
`student`	`contributions:write`, `requests:write`
Guest	No token and no write scopes.

The final code shall define these names once per backend security module and document the matching frontend type/guard names. A frontend role check never replaces backend authorization.

Visibility and status rules

Record	Initial status	Public visibility	Allowed transition
Core Material	Not applicable	Always visible while the record exists.	Admin create/edit/delete.
Contribution	`pending`	Hidden.	Admin → `approved` or `rejected`; owner edits/deletes only while pending.
Contribution	`approved`	Visible.	No Student edits; Admin may delete under policy.

Record	Initial status	Public visibility	Allowed transition
Contribution	`rejected`	Hidden.	No Student edits; Admin may delete or apply the documented moderation policy.
Request	`open`	Visible under public Request policy.	Owner/Admin → `fulfilled`; owner/Admin delete under the approved deletion policy.
Request	`fulfilled`	Visible with fulfilled status.	Repeated fulfillment is idempotent or a documented conflict; it must not silently reopen the record.

Input validation

- Email values must use the backend's accepted email validation and be unique case-insensitively where supported.
- Passwords must meet the documented minimum length/complexity policy and must never be echoed back.
- Course codes must be non-empty and unique.
- Material type must be one of `slides`, `notes`, `past_paper`, `book`, or `other`.
- External links must pass URL syntax validation and use the permitted scheme, normally `https`. The MVP does not promise that the remote link will remain reachable.
- Titles and descriptions must be trimmed and satisfy documented length limits.
- Status values are server-controlled; clients may not create an already-approved Contribution or an already-fulfilled Request by modifying the request body.

3.5 Traceability and verification matrix

Requirement	PRD/SRS intent	Main implementation area	Minimum verification
FR-01	FR-1–FR-5	Auth router/service, User model, registration/profile UI	Register, duplicate email, role escalation, `/me`.

Requirement	PRD/SRS intent	Main implementation area	Minimum verification
FR-02	FR-2, NFR-1–NFR-2	JWT/security dependencies, AuthContext, route guards	Valid login, invalid token, expiry, missing scope, frontend 401.
FR-03	FR-7, FR-18	Course repository/service, public course pages	Guest list/detail, search, unknown course.
FR-04	FR-6	Admin course router/service/UI	Admin CRUD, duplicate code, dependency `409`, denial.
FR-05	FR-9, FR-11, FR-17	Public material queries, course detail UI	Approved-only visibility, type filter, empty state.
FR-06	FR-8	Core Material model/router/service/admin UI	Admin CRUD, invalid URL/course, denial.
FR-07	FR-10	Contribution create schema/router/service/student UI	Authenticated create, pending default, invalid input, Guest denial.
FR-08	FR-11–FR-12	Moderation router/service/admin queue	Pending queue, approve/reject, public visibility, denial.
FR-09	FR-13	Contribution ownership service/student UI	Owner edit/delete, non-owner denial, status boundary.
FR-10	FR-14	Request create router/service/student UI	Create open Request, validation, Guest denial.
FR-11	FR-15	Public Request query/UI	Guest list, status rendering, unknown course.
FR-12	FR-16 plus TDD request deletion	Request service/UI	Owner/Admin fulfill, non-owner denial, repeated action, delete policy.

4. Verification and release acceptance

4.1 Test strategy

The team shall use multiple levels of verification:

1. **Unit tests:** service-layer rules such as ownership, status transitions, course deletion protection, and password/JWT helpers.
2. **API integration tests:** FastAPI `TestClient` tests for successful flows, validation errors, 401/403 boundaries, and CRUD behavior.
3. **Frontend tests or repeatable manual checks:** forms, route protection, loading/error/empty states, responsive layout, and safe external-link behavior.
4. **End-to-end smoke run:** Guest browse → Student register/login → Student submit Contribution/Request → Admin moderate/fulfill → public visibility check.
5. **Deployment checks:** production frontend, API health, database connection, CORS, environment variables, and cold-start note.

4.2 Minimum role/permission test matrix

Operation	Guest	Student	Admin
View courses and public materials	Allow	Allow	Allow
Register/login	Allow	Allow	Allow login; no public Admin registration
Create/edit/delete courses	Deny	Deny	Allow
Create/edit/delete Core Materials	Deny	Deny	Allow
Submit Contribution	Deny	Allow	Allow only if the §2.7 decision is approved
Approve/reject Contribution	Deny	Deny	Allow
Edit/delete own pending Contribution	Deny	Allow owner only	Allow according to policy
Submit Request	Deny	Allow	Allow only if the §2.7 decision is approved
View Requests	Allow	Allow	Allow
Fulfill own Request	Deny	Allow owner only	Allow

4.3 Definition of Done

A requirement may move to Trello `Done` only when:

- its user story and confirmation criteria pass;
- backend authorization is tested, including relevant negative cases;
- the UI handles success, loading, empty, and error states;
- code is merged through a reviewed pull request;
- automated tests pass, or a documented manual check exists where automation is impractical;
- documentation, API contracts, and traceability are updated;
- no secret or personal data is committed; and
- evidence is attached to the card or linked from the release evidence page.

4.4 Demonstration sequence

The final demonstration should use seeded, non-sensitive accounts and follow this order:

1. Open the application as a Guest and search for a course.
2. Open the course detail and show Core Materials, approved Contributions, filters, and Requests.
3. Register a Student and log in.
4. Submit a Contribution and show that it is pending and not public.
5. Submit a Request and show it as open.
6. Log in as Admin and create/edit a course or Core Material.
7. Approve the Contribution and show it now appears publicly; reject another and show it remains hidden.
8. Fulfill the Student's Request.
9. Demonstrate a denied Student/Admin action and show the correct 401/403 behavior.
10. Show the deployed URL, architecture, test results, and known limitations.

Approval

This Software Requirements Specification has been reviewed by the PassDown project team and is submitted as the requirements baseline for the Software Engineering course.

Team Leader signature: _____ Date: _____

Instructor/evaluator signature: _____ Date: _____